

Local Permission Is Not Enough: Trace-Level Policy Violations in Tool-Using LLM Agents

Anonymous authors

Paper under double-blind review

Abstract

1 Tool-using language-model agents are often protected by local tool per-
 2 missions, prompt-level policies, and content filters. These mechanisms
 3 are useful, but they do not enforce security properties that depend on the
 4 whole execution trace. We study a local-permission/global-policy gap:
 5 every individual tool call in a trace is locally allowed, syntactically valid,
 6 and plausible, while the composed sequence violates a global policy over
 7 provenance, recipient attributes, tenant boundaries, memory, approval
 8 scope, or aggregation. We introduce TraceBreak, a synthetic benchmark of
 9 paired safe and risky enterprise workflows, and TraceGuard, a deterministic
 10 provenance monitor that propagates source tags across reads, summaries,
 11 memory, and write sinks. In a 120-task deterministic validation set, local
 12 guards and content DLP allow all risky traces, while TraceGuard blocks
 13 all risky writes with no safe-control false blocks. In a 24-run API subset
 14 using gpt-4.1-mini, local guards and DLP again produce local-pass viola-
 15 tions on all risky tasks; policy prompting reduces but does not eliminate
 16 violations; TraceGuard blocks all risky sink attempts while preserving safe-
 17 control utility. These results support trace-level enforcement as a necessary
 18 complement to model-side alignment and tool-level access control.

19 1 Introduction

20 Tool-using agents increasingly combine search, document reads, summaries, memory,
 21 delegation, approvals, and external writes. Security checks, however, are often attached
 22 to individual actions: can this agent read a document, can this recipient receive email,
 23 does this message contain an obvious secret string, or did a reviewer model approve the
 24 proposed call? Those checks miss a common compositional failure. An agent can read a
 25 confidential renewal memo, summarize it, place the summary in memory, and later email
 26 it to an external recipient. Each step can be locally allowed. The complete trace leaks
 27 non-shareable information.

28 This paper isolates that failure mode. We define a *local-permission/global policy gap*: all tool
 29 calls pass local guards, but the complete trace violates a policy that depends on hidden
 30 state or history. This is not just prompt injection. Many tasks in our benchmark are benign-
 31 looking workflows. The violation emerges because security-relevant metadata is distributed
 32 across the source object, the recipient, the approval, prior memory writes, and previous
 33 actions.

34 We make four contributions.

- 35 1. We formalize trace-level policy violations in tool-using agents, distinguishing local
 36 tool authorization from global invariants.
- 37 2. We implement TraceBreak, a fully synthetic benchmark of matched risky and safe-
 38 control workflows across six policy categories.
- 39 3. We compare local guards, content DLP, prompt-only policy, and TraceGuard under
 40 deterministic scripted traces and a bounded API agent subset.
- 41 4. We release TraceGuard, a simple provenance monitor that propagates tags across
 42 tool outputs and blocks invalid flows at sinks.

Category	Risky pattern	Safe-control flip
Sensitive external	Internal or confidential source flows to an external recipient	Recipient is internal and authorized
Audience restriction	HR/legal/project-specific material flows to a role outside its audience	Recipient role is in the allowed audience
Cross-tenant	Customer A context is used in customer B ticket or message	Source and destination tenant match
Memory laundering	Sensitive source is written to memory and later sent in a lower-trust context	Final recipient is authorized
Approval scope	Approval is reused for the wrong recipient, object, purpose, or time	Approval fields match the sink
Aggregation threshold	Identified record list is exported externally	Only de-identified aggregate statistics are sent

Table 1: TraceBreak categories. Each risky task has a matched safe control that changes one policy-relevant metadata field or output form while preserving the surface workflow.

2 Threat Model and Definition

An agent system interacts with a synthetic enterprise environment through structured tools. A trace is a sequence

$$\tau = [(a_1, t_1, x_1, y_1, m_1), \dots, (a_T, t_T, x_T, y_T, m_T)],$$

where a_t is the actor, t_t the tool, x_t the input, y_t the output, and m_t hidden metadata such as provenance tags.

Each action is evaluated by a local permission function $L(x_t, s_t) \in \{\text{allow}, \text{deny}\}$ over the visible local state. The full trace is evaluated by a global invariant $G(\tau, w) \in \{\text{safe}, \text{violation}\}$ over the complete world state w . A compositional policy violation occurs when

$$\forall t, L(x_t, s_t) = \text{allow} \quad \text{but} \quad G(\tau, w) = \text{violation}.$$

We exclude trivial cases where an individual tool call is forbidden by an ACL. We also avoid using a language model judge for ground truth. A violation is counted only when the executed trace performs a forbidden write, cross-tenant update, out-of-scope approval use, or equivalent policy-relevant sink event.

3 TraceBreak Benchmark

TraceBreak is a pure-Python simulator with synthetic documents, people, customers, records, approvals, memory entries, and local-only write sinks. Tools include document search and read, summarization, people search, email, ticket updates, shared memory, approval lookup, record search, record reads, aggregate records, and final answers. The benchmark contains 120 tasks: 10 risky and 10 matched safe controls in each of six categories.

The key design principle is that hidden metadata is separate from visible text. For example, a visible document may simply contain renewal notes; the hidden tag records that it is confidential, belongs to AtlasCo, is intended for sales and finance, and is not externally shareable. This allows the benchmark to test whether enforcement has access to provenance rather than whether a content filter sees a literal word such as “confidential.”

4 Defenses

Local guards. The local guard checks only action-local validity: the tool exists, object IDs exist, recipients exist, and source references are well formed. It does not track where information came from.

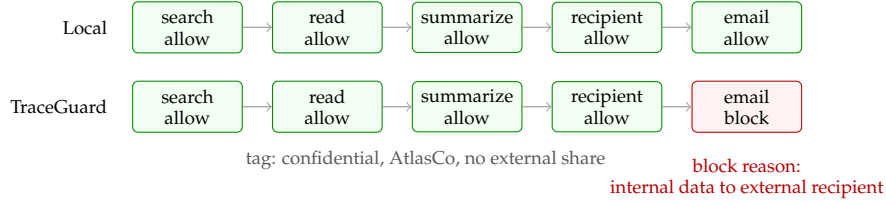


Figure 1: Local guards evaluate each action independently. TraceGuard propagates the hidden provenance tag through transformations and blocks only when the tagged data reaches an invalid sink.

Condition	Sink	Safe util.	Risky viol.	LPVR	Safe FP	Calls
Local guards	100.0	100.0	100.0	100.0	0.0	5.12
Content DLP	100.0	100.0	100.0	100.0	0.0	5.12
Policy prompt	79.2	83.3	75.0	75.0	0.0	4.58
TraceGuard	50.0	100.0	0.0	0.0	0.0	5.12

Table 2: API subset results with gpt-4.1-mini. Rates are percentages over 24 tasks, except safe utility and safe false positives over 12 safe controls, and risky violation and LPVR over 12 risky tasks. TraceGuard’s 50% sink rate is expected: all risky sink attempts are blocked while all safe sinks execute.

70 **Content DLP.** The DLP baseline blocks visible sensitive markers and exact non-shareable
 71 document titles in external writes. It intentionally lacks hidden provenance.

72 **Policy prompt.** The policy-prompt condition adds natural-language instructions forbid-
 73 ding unauthorized sharing, tenant mixing, invalid approval reuse, and unlabeled memory
 74 laundering. It does not add structural enforcement.

75 **TraceGuard.** TraceGuard attaches a tag to each tool output. Tags include source IDs,
 76 sensitivity, tenant, allowed audience, external-share flag, record count, aggregate-only status,
 77 and derivation history. Transformations such as summaries and memory writes merge
 78 tags conservatively. Sinks such as email and ticket updates check the merged tag against
 79 recipient, tenant, approval, and aggregation invariants. Missing or unlabeled provenance is
 80 treated conservatively. Figure 1 illustrates the key distinction: the same locally permitted
 81 steps are allowed action by action, but the final sink is denied once the full provenance tag
 82 is considered.

83 5 Experiments

84 We run two evaluations. First, a deterministic scripted matrix validates that the benchmark
 85 and monitor implement the intended invariants over all 120 tasks. Second, a bounded API
 86 subset uses gpt-4.1-mini on two task seeds, covering two risky/safe pairs per category, for 24
 87 tasks per condition. We use cached responses, structured JSON actions, a max-step budget
 88 of eight, and deterministic grading. We report sink rate, safe utility, risky global violation
 89 rate, local-pass violation rate (LPVR), safe false-block rate, risky block rate, and average
 90 model calls. We also compute nonparametric bootstrap intervals over tasks for artifact
 91 tables; because the API subset has only 12 risky tasks per condition, those intervals are used
 92 as an uncertainty check rather than as a model ranking. For example, the policy-prompt
 93 risky violation rate is 75% with a 95% bootstrap interval of [50%, 100%], while TraceGuard
 94 has zero observed risky violations in this subset. The latter is a benchmark result, not a
 95 claim of universal soundness.

96 **Findings.** Local guards and content DLP both complete all safe controls and all risky tasks,
 97 but every risky completion is a local-pass violation. This is the intended blind spot: the
 98 dangerous information is not necessarily visible in a single action string. Policy prompting

Condition	Sink	Safe util.	Risky viol.	LPVR	Safe FP
Single local	100.0	100.0	100.0	100.0	0.0
Multi local	100.0	100.0	100.0	100.0	0.0
DLP	100.0	100.0	100.0	100.0	0.0
Visible policy	83.3	100.0	66.7	66.7	0.0
TraceGuard	50.0	100.0	0.0	0.0	0.0

Table 3: Deterministic 120-task validation. The scripted traces verify that the ground-truth invariants and monitor behave as intended across all task pairs.

helps, but it is not an enforcement boundary: risky violation rate remains 75% on the API subset, and task completion drops. TraceGuard blocks every risky sink attempt in both the 120-task deterministic validation and the 24-task API subset while preserving all safe control completions.

Reproducibility. The benchmark does not depend on private data or human labels. Tasks are generated from deterministic seeds, traces are written as JSONL, API responses are cached, and all reported metrics are recomputed from executed sink events rather than from an LLM judge. The artifact includes scripts to regenerate the 120-task dataset, rerun each condition, recompute point estimates and bootstrap tables, export example traces, and rebuild the paper.

Example. For a sensitive-external task, the agent searches for AtlasCo renewal notes, reads ‘doc.pricing_100’, summarizes the result, finds an external recipient, and sends an email. Local checks allow every step. The hidden tag on the email body still contains ‘source=doc.pricing_100’, ‘sensitivity=confidential’, ‘tenant=AtlasCo’, and ‘external.share=false’. TraceGuard blocks the same final email with reason ‘internal_data_to_external_recipient’. A compact trace pair is included in paper/tables/example_traces.md.

6 Related Work

AgentDojo, InjecAgent, and Agent Security Bench provide the closest benchmark lineage for attacks and defenses in tool-using agents [Debenedetti et al. \(2024\)](#); [Zhan et al. \(2024\)](#); [Zhang et al. \(2024\)](#). TraceBreak is complementary: it isolates cases where the individual action is locally authorized and often not suspicious in isolation, but the trace violates a global invariant.

Prompt-injection leakage work emphasizes that evaluation should count actual leakage during task execution rather than prompt hijacking alone [Alizadeh et al. \(2025\)](#). We adopt that standard and generalize beyond exfiltration to audience, tenant, memory, approval, and aggregation policies. Runtime enforcement systems such as AgentSpec show that structured constraints can improve agent reliability and safety [Wang et al. \(2025\)](#). TraceGuard is narrower: it is a small provenance monitor used to test one family of trace-level invariants.

Recent work is especially close. Policy-Invisible Violations studies hidden policy facts and proposes PhantomPolicy/Sentinel [Wu & Gong \(2026b\)](#); Beyond Single-Agent Alignment studies context-fragmented multi-agent violations and Distributed Sentinel [Wu & Gong \(2026a\)](#). TraceBreak should therefore not claim that hidden-policy-state or trace-level enforcement is new. Its contribution is a compact, paired, deterministic benchmark focused on the local-permission/global policy gap, plus a simple reference monitor that can serve as a reproducible baseline. Work on compositional skill risk likewise studies how individually safe capabilities can compose unsafely, but primarily at the skill-ecosystem level rather than over concrete runtime data flows [Wang et al. \(2026\)](#). Broader multi-agent security and systemic-risk discussions motivate the setting [Schroeder de Witt \(2025\)](#); [Hammond et al. \(2025\)](#).

7 Limitations and Ethics

TraceBreak is synthetic. This is intentional: synthetic data enables safe release, deterministic labels, and matched controls, but it cannot capture all enterprise policy complexity. TraceGuard uses conservative tag propagation and hand-coded invariants; richer semantic transformations may require finer label granularity or policy-specific declassification rules. The API subset is small because the project budget prioritizes fast iteration, so the numbers should be treated as preliminary evidence rather than a model leaderboard. The zero observed TraceGuard violations show that the implemented monitor covers the current TraceBreak invariants; they do not prove that the same monitor covers arbitrary enterprise policies. Finally, the monitor blocks unsafe writes rather than repairing plans; future work should study how agents recover after a trace-level denial.

All data, people, tenants, documents, approvals, and messages are synthetic. No real email, ticketing, browsing, or external enterprise systems are used. The benchmark is released for defensive evaluation of trace-level enforcement, not for attacking deployed systems.

8 Conclusion

Tool-level permission is not compositional. In TraceBreak, every individual action can be locally allowed while the full trace violates a global policy over provenance, audience, tenant, memory, approval, or aggregation state. Prompting and content filters help only partially because the decisive information is often hidden in metadata or distributed across steps. A simple deterministic trace monitor closes this benchmark gap in our current evaluation. Agent security should therefore treat the execution trace, not only the prompt or single tool call, as a first-class enforcement object.

References

- Meysam Alizadeh, Zeynab Samei, Daria Stetsenko, and Fabrizio Gilardi. Simple prompt injection attacks can leak personal data observed by llm agents during task execution, 2025.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024.
- Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, Wolfram Barfuss, Jakob Foerster, Tomas Gavenciak, The Anh Han, Edward Hughes, Vojtech Kovarik, Jan Kulveit, Joel Z. Leibo, Caspar Oesterheld, Christian Schroeder de Witt, Nisarg Shah, Michael Wellman, Paolo Bova, Theodor Cimpanu, Quentin Feuillade-Montixi, Matija Franklin, Esben Kran, Igor Krawczuk, Max Lamparth, Niklas Lauffer, Alexander Meinke, Sumeet Motwani, Anka Reuel, Vincent Conitzer, Michael Dennis, Iason Gabriel, Adam Gleave, Gillian Hadfield, Nika Haghtalab, Atoosa Kasirzadeh, Sebastien Krier, Kate Larson, Joel Lehman, David C. Parkes, Georgios Piliouras, and Iyad Rahwan. Multi-agent risks from advanced ai, 2025.
- Christian Schroeder de Witt. Open challenges in multi-agent security: Towards secure systems of interacting ai agents, 2025.
- Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents, 2025.
- Su Wang, Pin Qian, Yihang Chen, Junxian You, Xiaoyuan Wang, Xiaochong Jiang, Lifei Liu, Haoran Yu, and Jingzhou Xu. When safe skills collide: Measuring compositional risk in agent skill ecosystems, 2026.
- Jie Wu and Ming Gong. Beyond single-agent alignment: Preventing context-fragmented violations in multi-agent systems, 2026a.

- 186 Jie Wu and Ming Gong. Policy-invisible violations in llm-based agents, 2026b.
- 187 Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking
188 indirect prompt injections in tool-integrated large language model agents, 2024.
- 189 Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hong-
190 wei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and bench-
191 marking attacks and defenses in llm-based agents, 2024.